

SimulEDA P2

Obs: As bibliotecas utilizadas estão disponíveis no arquivo .zip enviado junto ao simulado.

1 - Considere uma **Árvore B de grau $d = 2$** com os seguintes valores na raiz: 22, 50.

a) Faça a inserção incremental dos elementos, explicitando onde ocorrem as divisões: 60, 74, 100, 80, 37, 76, 67, 70, 61, 73, 28, 1, 12, 27, 24.

b) Faça a remoção dos elementos, explicitando os casos: 37, 1, 73

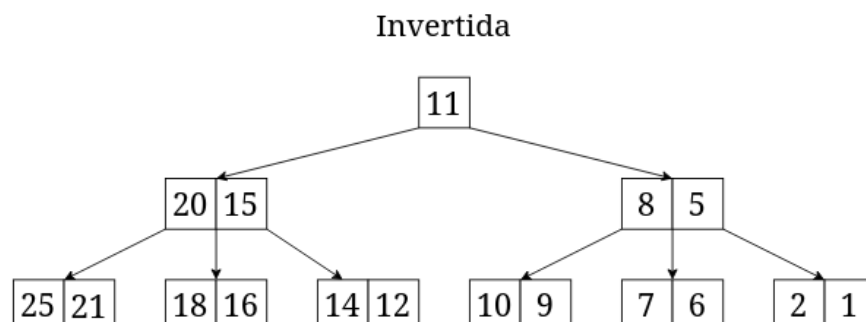
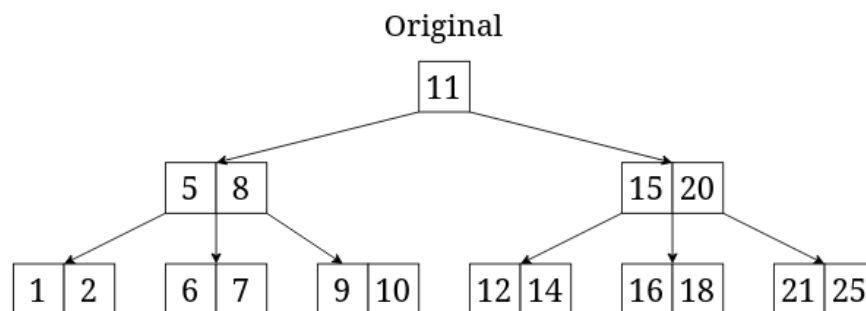
2 - Considere uma **Árvore B+ de grau $d = 2$** com os seguintes elementos na raiz: 15, 42

a) Faça a inserção incremental dos elementos: 88, 7, 63, 29, 91, 55, 12, 34, 76, 50, 5, 23, 68.

b) Faça a remoção dos elementos: 50, 91, 88, 34, 29, 55, 42, 68.

3 - Faça uma função que inverte uma árvore B.

TNoB* `inverte(TNoB *a);`



4 - Faça uma função que, dada uma **Árvore B**, retorne uma lista encadeada em ordem **DECRESCENTE**.

```
TLista* arvore_pra_lista(TNoB *a);
```

5 - Faça uma função que ao receber uma **Árvore B+** e um inteiro **x**, retorne uma string contendo o caminho entre a raiz e o nó folha que contém **x**.

```
char* caminho(TNoBM* a, int x);
```

Obs: Preencha o caminho com o número do filho(no vetor de filhos "p") em que x pode estar.

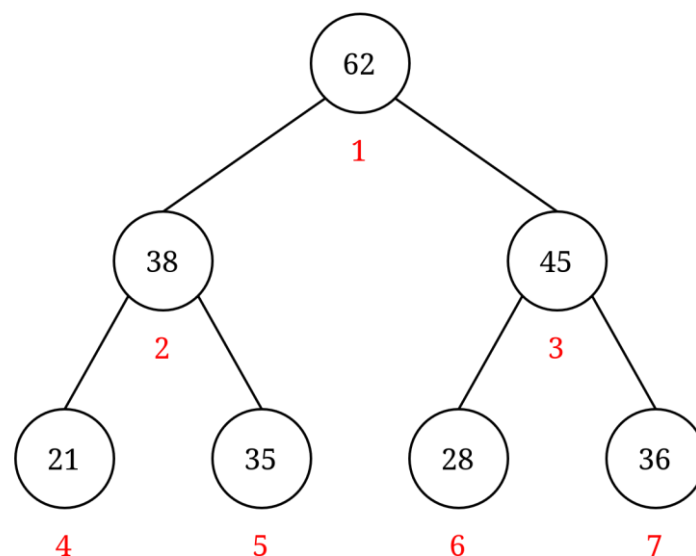
6 - Dado um vetor de inteiros repetidos entre 0 e N, faça uma função usando tabela hash que retorne a moda do vetor.

```
int moda(int *vet, int tam_vet, int n);
```

7 - Dados dois arquivos texto contendo inteiros, faça uma função que escreva em um arquivo de saída C todos os pares de números do arquivo A e B, que respeitem o critério: $a \% k + b \% k = k$

```
void soma_k(char *arqA, char *arqB, char *saida, int k);
```

8 - Considere a heap a seguir, faça as operações de forma incremental:



a) Faça a inserção incremental dos números a seguir: 15, 40, 69

- b) Diminua a prioridade do nó 3 para 20 e em seguida o nó 1 para 2.
- c) Aumente a prioridade do nó 9 para 51 e em seguida o nó 10 para 8001.
- d) Faça as duas remoções do elemento mais prioritário em sequência.

9 - Implemente a biblioteca de heap para heap n-ária (possui n filhos).

int pai(int n, int indice_filho);

int filho_i(int n, int indice_pai, int i);

void subir(int *heap, int n, int i);

void descer(int *heap, int ultimo_indice, int n, int i);

int insere(int *heap, int ultimo_indice, int n, int novo);

int exclui(int *heap, int ultimo_indice, int n);